

React avanzado

Módulo 3 – Unidad 3

Mi app con Contexto Global



Objetivos

Utilizar **Context API** para compartir y consumir información global en una aplicación React, evitando el *prop drilling* y aplicando buenas prácticas.



Consigna

1. Definir el caso de uso.

- Elegir una información que sea útil compartir en toda la app.

Ejemplos:

- Usuario autenticado.
- Tema (claro/oscuro).
- Idioma.

- Carrito de compras.

2. Planificar el contexto.

- Decidir:
 - Nombre del contexto (ej. **UsuarioContext**).
 - Datos que almacenará.
 - Funciones que modificarán esos datos.

3. Crear el *provider*.

- Definir un componente que envuelva toda la aplicación y provea el valor del contexto.
- Incluir el estado y las funciones que manipulen ese valor.

4. Consumir el contexto.

- Elegir al menos dos componentes en distintas partes del árbol.
- Usar el contexto para:
 - Mostrar un valor global (ej. nombre de usuario).
 - Modificar un valor global (ej. cambiar idioma o tema).

5. Prueba funcional.

- Modificar el valor en un componente y comprobar que el cambio se refleja en todos los que consumen el contexto.
- Verificar que al remover el *provider*, el `useContext` devuelva `undefined`.

6. Buenas prácticas.

- Usar un nombre semántico para el contexto y el *provider*.
- Evitar almacenar datos innecesarios en el contexto.
- Mantener cada contexto en un archivo independiente.

Formato de presentación

La entrega debe hacerse en un **repositorio en GitHub** que incluya:

- Proyecto en **React (Vite recomendado)**, limpio.
- **Contexto**: archivo del contexto y Provider con estado y funciones en `value` (opcional `useMemo`).
- **Consumo en al menos dos componentes** usando `useContext`.
- **Ejemplo de uso**: mostrar y modificar un valor global (usuario, tema, idioma o carrito).
- **Prueba funcional**: cambios reflejados en todos los consumidores; verificar qué pasa sin Provider.

- **README.md** con: descripción del caso, instrucciones de instalación/ejecución, capturas o GIF, créditos y bibliografía.

Criterios de evaluación

- Implementación correcta de **Context API**: creación del contexto, **Provider** y consumo con `useContext`.
- **Evitar prop drilling** y ubicar el **Provider** en el nivel adecuado.
- Diseño del **valor del contexto**: solo datos y funciones necesarias, nombres semánticos.
- **Organización del código**: contexto y **Provider** en archivos independientes; componentes consumidores claros.
- **Performance básica**: uso de `useMemo` (o patrón equivalente) para evitar renders innecesarios.
- **Prueba funcional** completa (actualización reflejada en múltiples componentes).
- Repositorio prolijo: estructura clara, README completo y capturas.



Bibliografía utilizada y sugerida

Libros y otros manuscritos

Banks, A. y Porcello, E. *Learning React: Modern Patterns for Developing React Apps*. 2ª ed. O'Reilly Media; 2020.

Artículos y documentación en línea

React. (s.f.-a). *Passing Data Deeply with Context*.

<https://react.dev/learn/passing-data-deeply-with-context>

React. (s.f.-b). *useContext*. <https://react.dev/reference/react/useContext>

React. (s.f.-c). *useMemo*. <https://react.dev/reference/react/useMemo>